1. CUSTOMER OBSESSION

Main Story CO-1

Tell me about a time you went above and beyond for a customer.

Use the **same STAR story** for:

- Tell me about one of your projects where you put the customer first.
- Tell me about a time you prioritized the customer's needs.
- Tell me about a time you went above and beyond for a customer.
- Tell me about a time you prioritized the customer's long-term interest over an immediate business goal.
- Tell me about a time you made something much simpler for customers.
- Tell me about a time you identified an opportunity to improve the customer experience.

STORY:

CO-1 — Final Draft

Situation:
During my Web Developer internship at LST Capital, team was working with a client on a new system where entire new household services booking application was needed to be developed. Client had already shared their initial requirements and scheduled a meeting to walk them through the proposed workflow and architecture. Our tech lead was supposed to lead that discussion, but shortly before the meeting, he became unavailable due to an unexpected personal situation.

The easiest option for us internally would have been to postpone the meeting. But from the client's perspective, that would have delayed their planning and potentially the overall project timeline.

Task:
I decided to step in and take ownership of the architecture discussion even though presenting the complete system design wasn't originally my responsibility. My goal wasn't simply to keep the meeting on the calendar; I wanted to make sure the client felt that we understood their actual needs and that the proposed system would be simple for them to use and maintain.

Action:
I first went back through the client's requirements and notes rather than relying only on our existing technical assumptions. I broke their requirements into smaller workflows and mapped each one to a specific system component.

While doing that, I also tried to look at the system from the client's perspective rather than purely from an engineering perspective. Wherever the workflow seemed unnecessarily complicated, I simplified how the components interacted and organized the presentation around the client's business flow instead of overwhelming them with implementation details.

Because I was taking over a discussion that would normally have been led by someone more senior, I also reviewed my proposed architecture and assumptions with my manager before the meeting. That helped me validate the technical feasibility while still keeping the customer's requirements at the center of the design.

During the client meeting, instead of simply presenting a finished architecture, I walked them through the workflow step by step, explained how product will be designed page by page and step by step so that they would have a user and business based clarity. That allowed us to resolve concerns immediately rather than discovering them later during development and increasing workload.

Result:
The client approved the proposed direction and was comfortable moving forward with development despite the unexpected absence of our tech lead. More importantly, we avoided delaying the customer because of an internal issue, and the clearer workflow reduced ambiguity going into implementation.

The project stayed on its planned timeline and ultimately met the client's expectations. Also we were able to deliver the entire project 2 week before the actual deadline after proper testing.

Main Story CO-2

Tell me about a difficult customer situation and how you handled it.

Same answer can cover:

- Tell me about a time you had to deal with a difficult customer.
- What is the most difficult customer situation you've had?
- Tell me about a time when you couldn't meet a customer's demand.
- Tell me about a time you said no to a customer request. Why?
- How did you handle a customer whose expectations you couldn't meet?

Situation:
While working on **Chat2SQL**, a database query assistant, one stakeholder wanted the system to make the experience as fast as possible by taking a natural-language question, generating SQL, and **executing that SQL automatically**.

From the customer's perspective, that made sense because it removed extra steps. But I was concerned that an AI-generated query could be syntactically valid while still referencing the wrong table or column, returning misleading results, generating an unnecessarily expensive query, or even modifying data if write operations were allowed.

Task:
My responsibility was to preserve the convenience the customer wanted without introducing unnecessary risk to their database.

I had to push back on the original request, but I didn't want to simply tell the stakeholder, “No, we can't do that.” I wanted to propose an alternative that still solved their underlying need for speed.

Action:
I explained the risks using concrete examples of how a generated query could look correct but still produce the wrong result because of schema ambiguity or incorrect relationships.

I then proposed a safer workflow:

Natural-language request → SQL generation → schema validation → query preview with explanation → user confirmation → execution.

I added validation to verify that referenced tables and columns actually existed in the selected schema, and I restricted the automated workflow to **read-only operations** so the assistant could not accidentally modify or delete production data.

I also included a structured explanation alongside the generated SQL so the user could understand what the query was doing rather than blindly trusting the model output.

The stakeholder initially preferred full automation because it was faster, but after I walked through the potential failure scenarios and showed that the validation layer only added a small amount of friction, they agreed that the safer workflow was better for long-term use.

Result:
We kept the convenience of natural-language SQL generation while adding safeguards that made the product much more trustworthy.

The final system was able to support **10+ database schemas** with validated SQL generation and structured summaries, while reducing the likelihood of manual-query errors.

The main lesson I took away was that **Customer Obsession doesn't always mean saying yes to exactly what the customer asks for**. Sometimes it means understanding the outcome they actually want, respectfully pushing back when the requested solution creates unnecessary risk, and finding a safer way to achieve the same goal.

3. BIAS FOR ACTION

Main Story BFA-1

Tell me about a time you had to make an important decision with incomplete information.

This is a huge cluster. Covers:

- Tell me about a time you had to make an urgent decision without data.
- Tell me about a time you didn't have enough data and had to use judgment.
- Tell me about a time you worked with incomplete information.
- Tell me about a decision based on limited information.
- Tell me about a time you took an important decision without enough data.
- Have you ever had several options but not enough time to investigate each?
- Tell me about a time you made a hard decision.
- Tell me about a time you made a decision without consulting your superior.
- Tell me about a time you had to act before you had all the answers.

Situation:
While working as a Software Engineer at USC Dornsife, we were building an AI-based learning platform. The initial product vision included avatar-generated video lectures and a basic LLM playground.

After speaking with friends and students who regularly used AI tools, I noticed a common problem: students could easily get an answer from one model, but they often did not understand **why the answer was good, what might be missing, or how a different model might approach the same problem**.

Task:
I wanted to make the LLM playground more focused on actual learning rather than simply generating answers.

I proposed turning it into a **multi-model comparison and evaluation platform**, and I took responsibility for designing and implementing that experience end-to-end.

Action:
Instead of allowing students to interact with only one model, I designed the system so that a single prompt could be sent to multiple models—currently including **GPT, Claude, Gemini, and Qwen**—with all of the responses displayed together on one screen.

I then introduced an **orchestration and judging layer**. The orchestrator manages the different model calls, while short-form and long-form judges evaluate the responses across factors such as correctness, reasoning, completeness, and relevance.

Building this required solving several technical challenges. Different providers returned incompatible response formats, so I created a common response structure. Calling multiple models also introduced significant latency, so I used concurrent processing rather than waiting for responses sequentially.

I also added safeguards for provider failures and timeouts so that one unavailable model would not break the entire comparison experience. Another challenge was judge consistency and potential bias, so I worked on structuring the evaluation prompts and outputs to make the comparisons more consistent and interpretable for students.

Throughout the process, I kept coming back to the original student pain point: the goal was not to expose more AI models, but to make it easier for students to **compare reasoning, understand differences, and learn from those differences**.

Result:
The playground evolved from a basic chatbot interface into a learning environment where students could compare different reasoning approaches, evaluate model responses, and understand why one answer might be stronger than another.

The feature is now used by approximately **350 students daily**, and the broader orchestration infrastructure handles thousands of model jobs.

This multi-model comparison and evaluation system, along with one other core product feature, also became **one of the two strongest features of the platform and received strong support from the VC backing the project**, which gave us additional validation that the idea had value beyond the initial student use case.

4. HAVE BACKBONE; DISAGREE AND COMMIT

Main Story HBD-1

Tell me about a time you disagreed with a teammate and how you resolved it.

Covers:

- Tell me about a conflict with someone on your team.
- Tell me about a time your teammate didn't agree with you.
- Tell me about a time your team members didn't agree with you.
- Tell me about a time you had conflicting ideas with teammates.
- Talk about conflict with coworkers.

- Tell me about a time people on your team disagreed with your approach.
- Tell me about a time your team didn't support your proposed solution.

Situation:
During my Full Stack Developer internship at Aasma Technology Solutions, our team was building a MERN-based product with several tightly coupled workflows. Two modules—inventory management and finished-product processing—were causing recurring bugs, and because everything was part of a monolithic application, debugging was becoming slow and difficult.

I proposed moving those problematic areas toward a microservices-based design, but my manager disagreed because he was concerned about migration complexity and the risk of missing delivery deadlines.

Task:
My goal was to improve the architecture without creating unnecessary delivery risk, while also understanding my manager's concerns instead of simply pushing my original idea.

Action:
I first analyzed the recurring issues and found that most of them were concentrated in those two modules.

Instead of recommending a complete rewrite, I proposed an incremental approach: we would continue building new features using the more modular architecture, while gradually migrating the older monolithic components between sprints.

I also built a small proof of concept around one problematic workflow to demonstrate how isolating the module could simplify debugging. I walked my manager and team through it, listened to their concerns around deployment complexity and maintenance overhead, and incorporated that feedback into the phased migration plan.

Result:
The team agreed to the phased approach, and we improved modularity without disrupting ongoing feature development. Although we were initially behind schedule because of the debugging issues, the new approach helped us catch up over the next **two sprints** and still meet our committed project deadline.

The experience taught me that disagreement is not about proving that my original idea is right. It is about using evidence, listening to valid concerns, and finding a solution that balances long-term technical improvements with the team's immediate priorities.

10. HIRE AND DEVELOP THE BEST

For SDE1, translate these into **peer/team mentoring**, not management.

Main Story HD-1

Tell me about a time you helped a struggling teammate succeed.

Covers:

- Tell me about a time a teammate was struggling.
- Tell me about a time you stepped in to help someone.
- Give me an example of helping a peer.
- Tell me about a time you helped someone in their work.
- Tell me about a time you went out of your way to help someone.
- How would you help a new employee having technical difficulties?

Situation:
PiXMod was a four-person team project where we built an AI-powered image editing platform with four main features: **transformer-based image retrieval, image segmentation, background removal and replacement, and image enhancement/editing**.

We divided the work across backend, frontend, and integration. Two of us, including me, mainly worked on the backend, while another teammate was responsible for integrating those backend workflows with the frontend.

By the third milestone, most of my backend work was complete, but the teammate handling integration had fallen behind because he had been out for about **three days with a high fever**. With the final deadline approaching, he was overwhelmed because he had to understand several different backend flows and integrate them quickly.

Task:
Even though integration wasn't my assigned responsibility, I wanted to help him catch up without simply taking over his work. My goal was to give him enough understanding of the system so he could complete the remaining integrations independently.

Action:
I arranged a working session with him and walked him through the data flow of all four major features—how an image moved from the frontend to the backend, through the relevant AI-processing pipeline, and then back to the UI.

Instead of coding everything for him, I **pair-programmed with him on one complete integration** so he could understand the pattern. We then broke the remaining integrations into smaller tasks, and I created a simple reference for the API inputs, outputs, and expected data flow.

After that, I stayed available for quick debugging sessions whenever he was blocked, while still letting him own the remaining implementation.

Result:
He was able to catch up and complete the integrations, and because the rest of the team coordinated around the remaining work, we finished the project **two days before the final deadline**.

That gave us extra time to perform proper end-to-end testing, complete the documentation, and prepare the final presentation.

The biggest thing I learned was that helping a teammate succeed isn't about doing their work for them. It's about understanding what is blocking them, giving them the right context and support, and helping them become productive independently.

11. FRUGALITY

Main Story FR-1

Tell me about a time you delivered something despite having very limited resources.

Covers:

- Tell me about a project with limited budget/resources.
- Tell me about working under tight deadlines with limited resources.
- Tell me about accomplishing more with fewer resources.
- Tell me about a project where you didn't have enough people.
- What did you do when you weren't given the resources you thought you needed?
- How do you manage limited resources?

Situation:
During my Software Engineer internship at Grad8, we were building an AI-powered grading platform to solve a resource problem for educators. For large classes, instructors often needed multiple graders to manually evaluate assignments, which increased cost, grading time, and sometimes introduced inconsistency between graders.

Our idea was to build a **rubric-based automated grading system with OCR**. An educator could upload an assignment or question paper, either generate an AI-based rubric or provide their own rubric, and then upload a ZIP containing the entire class's submissions. The system would extract each student's answers using OCR, grade them against the provided rubric, and generate personalized feedback for each rubric criterion.

At the prototype stage, however, we had only **two engineers** and approximately **$500 in AWS credits**, so we had to prove that the idea could work without simply solving performance problems by adding more infrastructure.

Task:
My responsibility was to help build the backend processing pipeline so that we could grade many student submissions efficiently while staying within our limited engineering and cloud resources.

I wanted the system to be scalable enough to demonstrate that educators could handle larger classes without proportionally increasing the number of graders.

Action:
I analyzed the grading workflow and found that several independent operations—such as processing different student submissions and rubric evaluations—were happening sequentially.

Instead of provisioning more compute, I redesigned parts of the Flask backend to process independent grading tasks concurrently. I also built the APIs in a modular way so that common functionality such as OCR processing, rubric evaluation, and feedback generation could be reused instead of creating separate implementations for every workflow.

Since there were only two of us, I also automated parts of the testing and deployment process so we could spend more of our limited development time improving the product rather than repeatedly handling manual releases.

Throughout the prototype, we monitored our AWS usage carefully so we could run realistic tests without exhausting the roughly **$500 credit budget**.

Result:
By using concurrency, reusable components, and automation rather than simply increasing infrastructure, we reduced grading turnaround time by approximately **60%** while staying within our prototype resource constraints.

More importantly, the product demonstrated how one educator could upload an entire class's submissions, apply a consistent rubric across every student, and receive structured AI-generated feedback without needing to proportionally increase the number of human graders.

This improved not only grading efficiency, but also **consistency and transparency**, because every student was evaluated against the same rubric and received criterion-level feedback explaining where they could improve.

The biggest lesson I took away was that **Frugality isn't simply about spending less—it is about designing systems that accomplish more with the resources already available.**

12. LEARN AND BE CURIOUS

Main Story LC-2

Tell me about something you learned from a failure or mistake.

Covers:

- Tell me about your biggest failure and what you learned.
- Have you ever failed? What did you learn?
- Tell me about a mistake that changed how you work.
- Tell me about something you learned after receiving feedback.

Situation:
While working as a Software Development Engineer at Aasma Technology Solutions, I was developing a cross-platform synchronization solution that integrated Jira with GitHub Actions.

For the initial implementation, I used timestamps to determine whether a record had changed and needed to be synchronized. The approach worked during normal testing, so I moved forward with it.

After the feature was integrated, the operations team working with our development team reported an issue: when **multiple updates happened within the same second**, only one of those changes could be captured correctly. I realized that my timestamp-based logic had an edge case I had not considered.

Task:
Since I had designed that synchronization logic, I took responsibility for fixing it. My goal was not just to patch the immediate issue, but to understand why my original design had failed and make the synchronization reliable for rapid consecutive updates.

Action:
I went back to the Jira and GitHub API documentation and reviewed how individual updates were represented.

That is where I discovered that the API records also had **unique identifiers**, which gave me a much more reliable way to distinguish individual events than relying only on timestamps.

I redesigned the synchronization logic so that, instead of only checking the latest timestamp, the system also tracked the last processed unique ID. This allowed us to distinguish multiple changes even when they occurred within the same second.

Before implementing it, I discussed the revised approach with my manager to make sure there were no additional integration risks. I then implemented the change and added targeted unit and integration tests for rapid consecutive updates and other edge cases before merging it.

Result:
The updated logic resolved the missed-update issue and restored reliable synchronization between the two platforms.

More importantly, the mistake changed how I approach integrations. I learned not to assume that the most obvious field—like a timestamp—is sufficient for identifying changes. Since then, I spend more time validating API guarantees, thinking through concurrency and edge cases, and adding tests for failure scenarios before merging a feature.

GEN AI

1. How do you use GenAI in your development workflow?

> "I treat GenAI as a collaborative assistant and productivity accelerator, not as a source of truth.
>
> I first understand the problem myself and form an initial idea of what the solution should look like. Then I use GenAI to challenge my thinking—whether I missed a requirement, an edge case, or a better design alternative.
>
> During implementation, I use it for things like brainstorming, boilerplate, debugging, first-draft unit tests, understanding unfamiliar APIs, and documentation.
>
> Once I have a candidate solution, I review the code myself, test it with different inputs and edge cases, run unit and integration tests, and verify unfamiliar APIs against official documentation.
>
> So AI helps me move faster, but I own the final technical decision and the code that gets shipped."

Key line to remember

“I use GenAI as a productivity accelerator, not as an authority.”

---

2. Which AI tools have you used?

> “I've worked extensively with multiple AI platforms including GPT, Claude, Claude Code, Gemini, Codex, and coding environments such as Cursor. I've also experimented with tools like Devin and Bolt during hackathons and side projects.
>
> I use different tools for different purposes—brainstorming, debugging, code generation, test generation, understanding unfamiliar code, and comparing alternative solutions.
>
> One thing I've learned from working with multiple models is that I don't choose a model simply out of habit. I look at the task, the quality of its output, latency, context requirements, and how easily I can validate the result.
>
> Ultimately, regardless of which model I use, I validate the output before making the final decision.”

Don't spend 30 seconds listing products. **Tool → purpose → lesson.**

---

3. Tell me about a real project where you used GenAI.

USC Dornsife Multi-Model Platform

> “One of the most impactful GenAI projects I've worked on is an AI-powered education platform at USC Dornsife.
>
> Initially, the product had the idea of a basic LLM playground, but I noticed that giving students one AI-generated answer didn't necessarily help them learn. They often couldn't tell whether an answer was complete or how another model might reason differently.
>
> I proposed and implemented a multi-model comparison system where a single prompt could be sent concurrently to GPT, Claude, Gemini, and Qwen, and the responses were displayed side by side.

I also built an orchestration and judging layer that evaluates responses across dimensions such as correctness, reasoning, completeness, and relevance.

Technically, I had to normalize different provider response formats, use concurrency to reduce latency, and handle failures and timeouts so one provider wouldn't break the whole workflow.

Today, that feature is used by approximately **350 students daily**, and the broader orchestration infrastructure handles thousands of model jobs."

Closing line

"That project taught me that calling an LLM API is relatively easy; building reliable orchestration, validation, and failure handling around it is the real engineering challenge."

3A. Follow-up: Why multiple models instead of just GPT?

"Because our goal wasn't simply to produce an answer. It was to help students understand different reasoning approaches.

Different models can approach the same question differently, so showing multiple responses allows students to compare those differences rather than blindly trusting one answer.

The judging layer then evaluates dimensions like correctness, reasoning, and completeness.

Architecturally, it also reduces dependence on a single provider. So the multi-model design supported both the educational goal and better system flexibility."

---

4. How do you validate AI-generated code or output?

"I never assume AI-generated output is correct.

First, I review the logic myself and make sure I can explain what the code is doing. If I can't explain it, I won't ship it.

Then I use deterministic validation wherever possible—compilation, type checking, linting, unit tests, integration tests, and edge-case testing.

If the AI is helping me with an unfamiliar API or framework, I verify its recommendation against the official documentation rather than trusting the generated explanation.

In my Dornsife project, we also have evaluation layers for comparing model responses, but I don't treat another LLM as an absolute source of truth either.

GenAI accelerates the implementation process, but the engineer still owns validation and correctness."

Very important sentence

> **"If I can't independently explain and validate AI-generated code, I don't ship it."**

4A. Follow-up: Can AI replace software engineers?

"I see AI as an engineering accelerator rather than a complete replacement.

It is becoming very good at generating implementation candidates, explaining code, and automating repetitive development work. But software engineering also requires understanding ambiguous requirements, making architectural trade-offs, evaluating risk, communicating with stakeholders, and ultimately being accountable for production outcomes.

AI can help with those decisions, but today the engineer still has to own them."

Keep this one short.

---

5. Tell me about a hallucination you encountered. How did you detect it?

This is the **concrete API example we locked**.

> “One example happened when I was integrating an external LLM provider API.
>
> I used an LLM to help draft the initial API call. The generated implementation looked completely reasonable, but when I ran it I consistently received a **400 Bad Request**.
>
> Instead of continuing to modify the code based on additional AI suggestions, I checked the provider's official API documentation.
>
> I discovered that the model had generated a request parameter that wasn't supported by that API version—it had essentially mixed conventions from another provider or version into the request.
>
> I corrected the implementation based on the official documentation, reran the tests, and the integration worked.
>
> That was a useful reminder that AI can produce code that looks extremely convincing even when one small technical assumption is completely fabricated.”

Closing line

> **“Plausible code is not proof of correct code.”**

That is probably your strongest line for this question.

5A. Follow-up: Why did the model hallucinate?

> “The model wasn't actually querying the provider's live API specification. It was generating the most probable answer based on patterns from its training data.
>
> Because different providers and even different versions of the same API can have similar request structures, it produced something that looked semantically reasonable but wasn't valid for the exact API version I was using.

That's why for rapidly changing APIs, I treat the model as a starting point and treat the official documentation as the source of truth."

5B. Follow-up: How did you fix it, and what did you change afterward?

"I fixed the immediate issue by comparing the generated request against the official documentation and replacing the unsupported parameter.

More importantly, I changed my workflow. For unfamiliar or rapidly changing APIs, I now verify the API contract first, then use GenAI to accelerate the implementation around that known contract.

I also make sure the integration has tests for successful responses, malformed requests, provider errors, and timeouts."

CUSTOMER OBSESSION

CO-3 — Customer asked for one thing but needed something else

Story tag: Chat2SQL — Safe SQL Execution

1. Customer asked for completely automatic SQL execution because they wanted speed.
2. You identified that their **actual need was fast database interaction**, not unrestricted execution.
3. Proposed read-only automation + schema validation + preview/confirmation for writes.
4. Customer got most of the speed benefit without exposing production data to unnecessary risk.

OWNERSHIP

OWN-1 — Took ownership outside your responsibility

Story tag: LST Capital — Tech Lead Unavailable

1. Tech lead unexpectedly unavailable before client meeting.
2. Presenting architecture wasn't your responsibility.
3. Reviewed requirements, validated architecture, stepped in and led client discussion.
4. Meeting wasn't delayed; client approved direction; project finished early.

OWN-2 — Took ownership of a critical issue

Story tag: Aasma — Jira/GitHub Timestamp Failure

1. Operations discovered rapid updates were being missed.
2. You had designed the timestamp logic, so you immediately owned the problem.
3. Read API docs → found unique IDs → redesigned synchronization → added edge-case tests.
4. Reliability restored; changed how you validate API assumptions.

OWN-3 — Decision without manager approval

⚠️ **NEW STORY NEEDED**

We don't currently have a clean story where you **actually acted without approval**.

Ideal flow:

1. Urgent situation + manager unavailable.
2. Explain what would happen if you waited.
3. Take a **reversible, limited-blast-radius decision** yourself.
4. Positive outcome + immediately communicate/document afterward.

FinNexus could be excellent here.

OWN-4 — Commitment at risk

Story tag: Aasma — Monolith → Phased Microservices

1. Recurring architecture/debugging problems put project deadline at risk.
2. You took responsibility for finding a path that protected delivery.
3. Instead of full rewrite, proposed phased migration and built POC.
4. Team recovered over two sprints and still met deadline.

BIAS FOR ACTION

BFA-2 — Calculated risk

Story tag: USC Dornsife — Multi-Model Platform

1. No large-scale evidence that students wanted multi-model comparison.
2. Waiting for perfect validation would delay experimentation.
3. Treated feature as reversible: collected qualitative feedback → prototype → validate.
4. Reached ~350 daily students and became a major product feature.

For **“risk that didn't pay off”**, switch to **Aasma Timestamp Failure**.

BFA-3 — Had to pivot quickly

Story tag: Aasma — Monolith → Microservices

1. Debugging overhead increased while deadline was approaching.
2. Continuing unchanged risked further delay.
3. Pivoted from full architectural replacement to phased extraction of problematic modules.
4. Team caught up within two sprints and delivered.

HAVE BACKBONE; DISAGREE AND COMMIT

HBD-2 — Disagreed with manager

Story tag: Aasma — Monolith → Microservices

1. You believed architecture was causing recurring problems.
2. Manager worried migration would jeopardize deadline.
3. Used evidence + POC rather than arguing from opinion.
4. Agreed on phased approach and delivered on time.

HBD-3 — Unpopular opinion / persuade others

Story tag: Aasma — Monolith → Microservices

1. Team's safer default was continuing with monolith.

2. You proposed changing architecture mid-development.
3. Analyzed recurring failures and demonstrated isolated workflow through POC.
4. Earned support for phased migration.

HBD-4 — Lost argument and still committed

Story tag: Aasma — Microservices, but PARTIAL

1. Your preferred broader migration wasn't accepted.
2. Manager's deadline/deployment concerns were valid.
3. You supported the compromise: phased migration rather than insisting on full redesign.
4. Committed fully and helped team deliver.

Important: if interviewer explicitly says *"you completely lost the argument"*, this isn't a perfect 10/10 match. A separate real incident would be stronger.

INVENT AND SIMPLIFY

IS-1 — Innovative solution

Story tag: USC Dornsife — Multi-Model LLM

1. Basic chatbot didn't sufficiently support learning.
2. Identified comparison/reasoning as the deeper student problem.
3. Built multi-model orchestration + judging rather than another single chatbot.
4. ~350 daily users + strong product validation.

IS-2 — Simplified complex system/process

Story tag: Grad8 — Automated Grading

1. Entire-class grading involved OCR, rubric evaluation and feedback.
2. Sequential workflow was slow and engineering resources were limited.
3. Created reusable modules + concurrent processing instead of separate implementations.
4. ~60% turnaround improvement within limited resources.

IS-3 — Multiple possible solutions

Story tag: Aasma — Microservices Decision

1. Options: stay monolithic / full rewrite / phased migration.

2. Full rewrite improved architecture but risked deadline.
3. Chose phased migration based on affected modules + POC.
4. Improved modularity while meeting delivery commitment.

DIVE DEEP

DD-1 — Root cause / difficult technical problem

Story tag: Aasma — Timestamp Sync Failure

1. Multiple changes in same second caused missing synchronization.
2. Reproduced issue and traced synchronization logic.
3. Read API contracts → timestamp wasn't unique → discovered event IDs.
4. Switched to unique-ID tracking + added targeted tests.

DD-2 — Used details/data to change decision

Story tag: Aasma — Microservices Investigation

1. Team initially saw recurring bugs as general development problems.
2. You examined where failures/debugging effort were actually concentrated.
3. Found inventory + finished-product modules were driving most issues.
4. That evidence changed proposal from full rewrite to targeted/phased migration.

DD-3 — Complex/unfamiliar system

Story tag: USC Dornsife — Multi-Model Architecture

1. Needed to integrate several different LLM providers.
2. Learned their APIs, response formats, failure behavior and evaluation issues.
3. Built provider abstraction + concurrency + timeout handling + judges.
4. Result became production-scale orchestration used daily.

ARE RIGHT, A LOT

AR-1 — Several solutions, use judgment

Story tag: Aasma — Microservices

1. Three options: do nothing / rewrite / phased migration.
2. Compare maintainability, migration risk and deadline.
3. Use POC + recurring bug evidence.
4. Select phased approach; deliver successfully.

AR-2 — Opinion differed from others

Story tag: Aasma — Microservices

1. Manager preferred avoiding architectural change.
2. You believed current architecture would create greater long-term delivery risk.
3. Demonstrated evidence instead of insisting you were right.
4. Compromise proved workable and project met deadline.

For **"I thought I was right but discovered I was wrong"**, use **Aasma Timestamp Failure** instead.

DELIVER RESULTS

DR-1 — Tight deadline

Story tag: PiXMod — Teammate Fell Behind

1. Integration teammate lost ~3 days near final deadline.
2. Integration became project bottleneck.
3. Pair-programmed one workflow + broke work into smaller tasks + API reference.
4. Finished **2 days early**, leaving time for testing/docs.

DR-2 — Commitment/deadline at risk

Story tag: Aasma — Microservices

1. Debugging issues had put team behind schedule.
2. Realized continuing unchanged could miss commitment.
3. Changed approach to phased modularization.
4. Recovered over two sprints and met deadline.

DR-3 — Significant setback

Story tag: PiXMod — Teammate Illness

1. Key teammate unavailable for three days.

2. Integration work accumulated close to deadline.
3. Helped unblock them without simply taking over.
4. Entire team still finished two days early.

DR-4 — Trade-off / reduce scope

Story tag: Aasma — Phased Microservices

1. Full architecture migration was desirable long term.
2. Full migration would create unacceptable deadline risk.
3. Reduced immediate scope to two problematic modules + future gradual migration.
4. Protected both quality and deadline.

DR-5 — Exceeded goal

Story tag: LST Capital — Client Project

1. Goal was successful client delivery on schedule.
2. Clarified requirements early and eliminated ambiguity.
3. Prevented rework through upfront customer alignment.
4. Delivered fully tested project **2 weeks early**.

THINK BIG

TB-1 — Challenged status quo

Story tag: USC Dornsife — Multi-Model Platform

1. Assigned direction was essentially an LLM playground.
2. Saw larger opportunity: teach students to compare reasoning.
3. Proposed orchestration + multiple models + judges.
4. Became one of platform's strongest features.

TB-2 — Significant accomplishment

Story tag: USC Dornsife — Multi-Model Platform

1. Started from ambiguous product concept.
2. Personally designed multi-provider architecture.
3. Solved normalization, latency, failures and evaluation.
4. ~350 daily students + thousands of model jobs.

TB-3 — Future scale

Story tag: USC Dornsife — Provider-Agnostic Orchestration

1. Didn't hard-code product around one LLM.
2. Created common model-response structure.
3. Added concurrent orchestration + independent provider failure handling.
4. Makes adding/swapping providers substantially easier as system scales.

HIRE AND DEVELOP THE BEST

HD-2 — Helped someone learn/grow

Story tag: PiXMod — Integration Teammate

1. Teammate lacked context across backend AI pipelines.
2. Don't do their work for them.
3. Pair-program one integration + explain pattern + give API reference.
4. They independently completed remaining integrations.

HD-3 — Strong teammate → even better

⚠️ **NEW STORY NEEDED**

Current PiXMod story is about someone **struggling**, not an already-high performer.

Ideal flow:

1. Strong teammate already performing well.
2. Identify one area where they could expand impact.
3. Give knowledge/opportunity/feedback rather than rescuing them.
4. Their impact or ownership increases afterward.

FRUGALITY

FR-2 — Simpler/cheaper solution

Story tag: Grad8 — Concurrency Instead of More Infrastructure

1. Only two engineers + ~$500 AWS credits.
2. Could have solved performance by buying more compute.
3. Reworked sequential pipeline using concurrency + reusable components.
4. ~60% faster while staying within resource constraints.

LEARN AND BE CURIOUS

LC-1 — Learned technology quickly

Story tag: USC Dornsife — Multi-LLM Integration

1. Needed to integrate unfamiliar model providers.
2. Studied APIs, response formats, limitations and failure patterns.
3. Built common interface + concurrency + timeout handling.
4. Knowledge became reusable orchestration infrastructure.

INSIST ON THE HIGHEST STANDARDS

HS-1 — Refused low-quality solution

Story tag: Chat2SQL — Safe Execution

1. Customer wanted fastest possible auto-execution.
2. Technically possible ≠ production-safe.
3. Added schema validation + read-only guardrails + preview/confirmation.
4. Preserved usability while raising safety/correctness standard.

HS-2 — Quality vs deadline

Story tag: Aasma — Phased Microservices

1. Architecture needed improvement, but deadline mattered.
2. Full migration risked missing delivery.
3. Refused both extremes: no change and full rewrite.
4. Phased approach preserved essential quality while meeting deadline.

HS-3 — What would you improve?

Story tag: Aasma — Timestamp Sync Failure

1. Original implementation worked for normal cases.
2. Missed same-second update edge case.
3. If repeating project, validate API uniqueness guarantees and concurrency cases before design.
4. Add failure/edge-case tests much earlier.

EARN TRUST

ET-1 — Significant mistake

Story tag: Aasma — Timestamp Sync Failure

1. Clearly say: **"I made a design mistake."**
2. Timestamp assumption failed under rapid updates.
3. Owned investigation and corrected root cause.
4. Added prevention mechanisms/tests and changed your engineering process.

ET-2 — Negative feedback

Story tag: Aasma — Timestamp Sync Failure

1. Operations/manager pointed out the implementation missed an important edge case.
2. Don't defend yourself—acknowledge their criticism was valid.
3. Investigate root cause and redesign.
4. Convert feedback into a permanent testing/API-validation habit.

ET-3 — Rebuild trust

Story tag: Aasma — Timestamp Sync Failure

1. Your implementation caused reliability issue.
2. Transparently own it rather than minimize it.
3. Explain root cause, fix and stronger testing.
4. Restore reliability and confidence through actions.

ET-4 — Needed help

⚠️ **NEW STORY NEEDED**

Ideal flow:

1. You encountered something outside your expertise.
2. Recognized trying alone would waste time/risk quality.
3. Asked the right expert a specific question, learned from them, then executed yourself.
4. Result + show that asking for help increased—not reduced—your ownership.

EARTH'S BEST EMPLOYER

BE-1 — Teammate struggling

Story tag: PiXMod — Integration Teammate

1. Teammate returned after illness and was overwhelmed.
2. Recognized human issue before just focusing on deadline.
3. Pair-program + break tasks down + stay available.
4. They regained independence and team delivered early.

BE-2 — Improved team environment/morale

Story tag: PiXMod — Integration Teammate

1. Deadline pressure + teammate overwhelmed.
2. Reduce stress by turning unclear backlog into manageable work.
3. Create documentation and supportive working sessions.
4. Team recovered and had extra time before submission.

BE-3 — Long-term growth

Story tag: PiXMod — PARTIAL / preferably use future HD-3 story

1. Don't simply complete teammate's tasks.
2. Teach system/data-flow pattern.
3. Give them ownership of remaining integrations.
4. They become independently productive.

SUCCESS AND SCALE BRING BROAD RESPONSIBILITY

SS-1 — Broader consequences

Story tag: Chat2SQL — Database Safety

1. Immediate feature goal = faster SQL execution.
2. Broader consequence = incorrect/destructive queries against real data.
3. Design guardrails instead of optimizing only immediate UX.
4. Safer system that still delivers user value.

SS-2 — Wrong direction / changed course

Story tag: USC Dornsife — Basic Playground → Learning Platform

1. Initial direction = basic LLM playground.
2. Student conversations showed answer generation wasn't the real problem.
3. Changed focus toward comparison, reasoning and evaluation.
4. Multi-model system became high-value feature.

Caution: only use this for “75% through” if the timing genuinely happened that late.

SS-3 — Decision you regret

Story tag: Aasma — Timestamp Sync Failure

1. Chose timestamp as event identity.
2. Didn't fully consider rapid concurrent updates.
3. Failure exposed flaw → redesigned with unique identifiers.
4. Learned to validate API guarantees and edge cases before implementation.

Only 3–4 genuine gaps remain

I would **not create more stories for everything above**. Your existing stories already adapt naturally.

The only ones I'd still look for are:

1. **OWN-3 — Independent decision without approval**

2. **ET-4 — Needed help**
3. **HD-3 / BE-3 — Developing an already-strong teammate**
4. **HBD-4 — Truly lost an argument and committed** *(optional; Aasma can survive this if necessary)*

Everything else can be answered using your existing portfolio with the four-point recall flows above.

PREDICTED

Behavioral / Leadership Questions

1. "Tell me about a time something you built failed."

Story: Aasma — Jira/GitHub Timestamp Synchronization

Situation:

"At Aasma Technology Solutions, I built synchronization logic between Jira and GitHub Actions. My original design used timestamps to identify new updates. It passed our normal testing, but after integration the operations team found that when multiple updates happened within the same second, one could be missed."

Task:

"I had designed that logic, so I took ownership of the failure. My goal was not just to patch the symptom but to understand why my assumption was wrong."

Action:

"I reproduced the edge case, went back through the APIs, and discovered that individual events had unique identifiers. I replaced timestamp-only tracking with logic that also tracked the last processed unique ID. I then added unit and integration tests specifically for rapid consecutive events."

Result:
"The missed-update issue was resolved and synchronization became reliable. The bigger lesson was that I had treated a timestamp as an identity guarantee when it wasn't one. Since then, I validate API guarantees explicitly and test concurrency and edge cases before shipping."

Trade-off: Simple timestamp logic was easier to implement, but unique-ID tracking added a little state management in exchange for much stronger correctness.

---

2. "Tell me about a time you had to finish something with insufficient resources."

Story: Grad8 — AI Grading Platform

Situation:
"At Grad8, we were building an OCR and AI-based grading platform, but at the prototype stage there were only two engineers and roughly $500 in AWS credits."

Task:
"My responsibility was to make the backend capable of processing an entire class's submissions without solving every performance problem by buying more compute."

Action:
"I profiled the workflow and found that independent student submissions and rubric evaluations were being processed sequentially. I redesigned those parts to run concurrently, created reusable OCR, grading, and feedback components, and automated parts of testing and deployment so our two-person team wasn't spending time on repetitive work."

Result:
"We reduced grading turnaround by roughly 60% while staying within our prototype resource constraints."

Trade-off: Instead of scaling vertically with more infrastructure, I accepted some additional concurrency complexity to dramatically improve resource efficiency.

---

3. "Describe a difficult bug that took a long time to identify."

Story: Aasma — Timestamp Synchronization Bug

> "A difficult bug I investigated involved Jira/GitHub synchronization. It was difficult because normal testing passed; the failure only appeared when two or more updates occurred within the same second.
>
> I initially used timestamps to identify new events. Once the issue appeared, I reproduced the rapid-update case, reviewed the API documentation, and discovered that timestamps weren't sufficient to uniquely distinguish events. The APIs exposed unique IDs, so I changed the design to track the last processed event ID as well.
>
> I then added targeted tests around rapid updates and other edge cases.
>
> The important lesson was that intermittent bugs often require challenging the assumptions underneath the implementation rather than adding patches around the observed symptom."

Trade-off: Slightly more state-management complexity for deterministic event identification.

---

4. “Tell me about a time you disagreed with someone about the correct technical approach.”

Story: Aasma — Monolith vs Microservices

Situation:

“Our MERN application had tightly coupled inventory and finished-product modules, and recurring bugs were making debugging increasingly expensive.”

Task:

“I believed isolating those components would improve maintainability, but my manager was concerned that a microservices migration could jeopardize our deadline.”

Action:

“I didn't argue from opinion. I examined where the recurring issues were concentrated and built a small proof of concept isolating one problematic workflow. I then proposed a compromise: don't rewrite everything. Move the problematic and new components toward a modular architecture while migrating older pieces gradually between sprints.”

Result:

“My manager and the team accepted the phased approach. We recovered from the earlier debugging delays over two sprints and still met the committed deadline.”

Trade-off: Full migration had greater long-term architectural cleanliness, while doing nothing was lower short-term risk. The phased approach deliberately traded architectural purity for manageable delivery risk.

---

5. “Tell me about a technically ambiguous decision where there was no obvious correct answer.”

Story: USC Dornsife — Multi-Model LLM Platform

Situation:

“We were building an AI learning platform, but the LLM playground was loosely defined. We didn't yet have large-scale user data showing exactly what students wanted.”

Task:

“I had to decide whether we should create another standard chatbot or build something more focused on learning.”

Action:

“I gathered qualitative feedback from students and noticed that the problem wasn't access to an answer; it was understanding whether an answer was strong and how different models reasoned. I chose to prototype a multi-model comparison system with GPT, Claude, Gemini and Qwen, plus an evaluation layer.”

Result:

“The feature grew to roughly 350 daily student users and the orchestration layer handles thousands of model jobs.”

Trade-off: Multi-model comparison increased cost, latency and engineering complexity versus a single-model chatbot, but it provided substantially more educational value and reduced dependence on one provider.

---

6. “Describe a time you learned something unfamiliar quickly.”

Story: USC Dornsife — Multi-Provider LLM Integration

“At USC Dornsife, I had to integrate several LLM providers that behaved differently. I couldn't treat GPT, Claude, Gemini and Qwen as interchangeable APIs.

I went through each provider's API contract, response format, timeout behavior and error handling, then built a common internal response structure so the rest of our product didn't need provider-specific logic.

I also learned how to issue the independent model requests concurrently and handle partial failures without allowing one unavailable model to fail the whole comparison.

The result was reusable orchestration infrastructure supporting thousands of model jobs.

My takeaway is that when I need to learn something quickly, I focus first on the interfaces and failure modes I actually need for the product instead of trying to master the entire technology upfront.”

---

7. “Tell me about a time you changed your view after receiving new information.”

Story: Aasma — Microservices Compromise

“I originally believed a broader move toward microservices was the right answer to our recurring architectural problems.

My manager pushed back because a larger migration created deployment and deadline risk. Instead of treating that as resistance, I considered the information he had that I was underweighting: migration complexity and our immediate delivery commitment.

I changed my position from pushing a broader migration to recommending a phased approach. We isolated the highest-problem areas first and migrated additional components gradually.

That approach still improved modularity while allowing us to meet our deadline.

I learned that being technically right about the long-term destination doesn't mean your original migration strategy is right.”

---

8. “Describe a situation in which you helped another engineer or teammate understand something.”

Story: PiXMod — Integration Teammate

Situation:

“In our four-person PiXMod project, a teammate responsible for integration missed about three days because of illness and returned with several backend workflows to understand.”

Task:

“I wanted to help them recover without simply doing their work for them.”

Action:

“I walked them through how data flowed across our retrieval, segmentation, background-removal and editing features. Then I pair-programmed one complete integration with them and created a small API input/output reference. After that, they owned the remaining integrations and I only helped when they were blocked.”

Result:
"They completed the integrations independently and the team finished two days before the deadline."

Trade-off: Taking over their work might have been faster for one day, but teaching the pattern created independence and reduced the risk of me becoming the integration bottleneck.

---

9. "Tell me about your most intellectually difficult project."

Story: USC Dornsife — Multi-Model Platform

> "The most intellectually difficult project I've worked on is the multi-model learning platform at USC Dornsife.
>
> The difficulty wasn't making one API call. I had four providers with different response formats and failure characteristics, multiple requests had to run concurrently, one provider couldn't be allowed to break the entire request, and then we had the additional problem of evaluating responses consistently.
>
> I created provider abstractions and a common response structure, used concurrent execution to avoid sequential latency, added timeout and partial-failure handling, and built structured judging flows around correctness, reasoning, completeness and relevance.
>
> The key trade-off was quality versus latency and complexity: querying more models produced richer comparisons but increased cost and response time.
>
> The system ultimately grew to roughly 350 daily student users and thousands of model jobs."

---

10. “Tell me about a time you took ownership of something outside your assigned responsibilities.”

Story: LST Capital — Client Architecture Meeting

Situation:
“At LST Capital, our tech lead unexpectedly became unavailable shortly before an important client architecture meeting.”

Task:
“Leading that meeting wasn't my responsibility, but rescheduling would have pushed the customer's planning because of an internal problem.”

Action:
“I stepped in. I reviewed the customer's requirements, mapped them into clear workflows, simplified the technical explanation around their business flow, and validated my architectural assumptions with my manager before presenting.”

Result:
“The client approved the direction, we avoided delaying them, and the completed product was ultimately delivered two weeks ahead of the planned deadline.”

Trade-off: I accepted additional preparation and personal responsibility rather than transferring the cost of our internal staffing issue to the customer.

---

11. “Tell me about a time you made a decision without waiting for permission.”

Important: don't force one of the existing stories.

Your current locked stories do **not** contain a clean, truthful example where you genuinely made a consequential decision without approval. For example, in Dornsife you aligned the idea before implementation, and at LST you validated the architecture with your manager.

For Amazon, prepare a real incident with this structure:

> **Situation:** “An urgent but bounded issue happened while the decision-maker was unavailable.”
>
> **Task:** “Waiting created more risk than acting.”
>
> **Action:** “I chose a reversible action with a limited blast radius, monitored it, documented my reasoning, and immediately informed the appropriate person afterward.”
>
> **Result:** “The issue was contained and the decision was later validated.”
>
> **Trade-off:** Explicitly compare **risk of acting** against **risk of waiting**.

Do **not** invent this one.

---

12. “Tell me about a disagreement with a teammate or manager.”

Story: Aasma — Monolith vs Microservices

> “I disagreed with my manager about whether we should change the architecture of a MERN system that had recurring bugs in two tightly coupled modules. I favored moving toward microservices; he was concerned about migration risk and deadline impact.

I analyzed where the problems were concentrated, built a POC showing the benefit of isolating one workflow, and then changed my proposal from a full migration to a phased one.

We both gave something up: I gave up the immediate full migration, while he accepted introducing architectural change during development.

The team adopted the phased design, recovered over two sprints and met the deadline.

I learned that disagreement is most productive when you bring evidence and remain willing to change the implementation path."

---

13. "Tell me about a time you had to defend an unpopular technical decision."

Story: Aasma — Microservices POC

"I proposed changing part of our architecture at a point when the safer organizational choice was to continue with the monolith.

I understood why it was unpopular: migrations create operational risk and we had a deadline.

Instead of repeatedly arguing that microservices were better, I gathered evidence showing that recurring failures were concentrated in two modules and built a small POC demonstrating how isolation improved debugging.

I then narrowed my proposal to a phased migration rather than insisting on a full rewrite.

The team accepted it and we ultimately met the project deadline.

The important part for me was defending the problem I believed needed solving while remaining flexible about the exact solution."

---

14. "Describe a project where you personally drove the outcome."

Story: USC Dornsife — Multi-Model Platform

"At USC Dornsife, I personally drove the multi-model comparison feature from identifying the problem through implementation.

I noticed that a basic LLM playground mostly gave students another place to get an answer. I proposed focusing on comparison and reasoning instead.

I designed the flow where one prompt reaches multiple models, built the orchestration and normalization layer, implemented concurrent provider calls and timeout handling, and contributed to the judging system used to compare responses.

I also kept the architecture centered on the underlying customer need—helping students understand differences rather than simply showing more models.

The feature now serves roughly 350 students daily and the broader orchestration system processes thousands of model jobs."

Technical Project Walkthrough + Follow-ups

Use **USC Dornsife** consistently for questions 15–19 so the interviewer gets one coherent technical story.

15. “Walk me through the most technically challenging project you have worked on.”

> “I'd choose the multi-model AI learning platform I worked on at USC Dornsife.
>
> The user sends one prompt, and our orchestration layer sends it concurrently to models including GPT, Claude, Gemini and Qwen. Each provider sits behind an adapter because their APIs and response formats differ. We normalize those responses into a common internal representation.
>
> We then run evaluation flows that compare answers across correctness, reasoning, completeness and relevance before presenting the comparison to the student.
>
> The difficult engineering problems were concurrency, provider-specific integration, partial failures, timeout handling and judge consistency.
>
> One important trade-off was that additional models improve comparison quality but increase latency and cost, so I avoided sequential calls and designed the flow so one failed provider wouldn't block everything.
>
> Today the feature supports approximately 350 daily students and thousands of model jobs.”

---

16. “What was your specific contribution?”

> “My specific contribution was the multi-model comparison and orchestration experience.
>
> I proposed the direction, designed how one prompt would fan out to multiple providers, normalized the incompatible provider responses into a common structure, used concurrency rather than sequential calls, and added failure and timeout handling so a single provider couldn't fail the entire comparison.
>
> I also worked on the structured evaluation layer used to compare responses across correctness, reasoning, completeness and relevance.
>
> So I wasn't simply integrating one LLM API; I owned a substantial part of the system around how multiple providers interacted reliably.”

---

17. “What trade-offs did you make?”

> “The biggest trade-off was **response quality versus latency, cost and complexity**.
>
> A single model would have been cheaper and simpler, but it wouldn't satisfy our learning goal of comparing reasoning approaches.
>
> Querying multiple models sequentially would have made latency unacceptable, so I used concurrency. That reduced waiting time but introduced more complicated timeout and partial-failure handling.

We also had to be careful with the judge layer. Adding more automated evaluation gives users more guidance, but an LLM judge is itself probabilistic, so I structured its output and never treated it as absolute ground truth.

Architecturally, I chose additional system complexity because it directly supported the educational value of the product."

---

18. "What would you change now?"

"If I redesigned it today, I would strengthen three areas.

First, I would build a formal evaluation dataset with representative student questions and expected characteristics so model and prompt changes could be regression-tested.

Second, I would add more intelligent model routing instead of always treating providers equally—for example choosing providers based on task, latency, cost or availability.

Third, I would strengthen observability around p95 latency, provider error rates, token cost and judge disagreement.

The original architecture optimized for proving the user value quickly. At larger scale, I'd shift more attention toward systematic evaluation, cost efficiency and operational visibility."

This is a **future-improvement answer**, not a claim that these mechanisms already exist.

---

19. "How did you measure whether it worked?"

“I looked at both product adoption and system behavior.

On the product side, the strongest outcome was usage: the feature reached approximately **350 students daily**, and the broader orchestration system handles thousands of model jobs.

I also cared about whether the design continued functioning when providers behaved differently, which is why timeout and failure isolation were part of the architecture.

If I were measuring the system at larger scale today, I would formalize p95 latency, provider failure rate, cost per request and judge agreement as additional technical KPIs.

So my definition of success wasn't simply that the API call worked—it was that students repeatedly used the experience and that the orchestration could handle real multi-provider workloads.”

Ambiguity Sequence

Questions 20–24 can all use **USC Dornsife**.

20. “Tell me about a time you were given an ambiguous problem and had to determine what to do.”

“At USC Dornsife, the requirement was essentially to create an AI learning platform with an LLM playground, but there wasn't a detailed specification for what the playground should actually do.

Rather than immediately building another chatbot, I tried to understand the underlying user problem.

Through conversations with students, I found that access to an answer wasn't the biggest gap; students struggled to judge whether an answer was strong or how alternative reasoning might look.

I proposed a multi-model comparison and evaluation experience and built the orchestration around it.

The trade-off was acting on qualitative evidence rather than waiting for perfect quantitative data. I treated the feature as reversible and validated the direction through actual usage.

It eventually reached roughly 350 daily students.”

---

21. “How did you gather information?”

“I started with the people experiencing the problem rather than jumping directly into implementation.

I spoke with students who regularly used AI tools and compared what they said with my own experience using those tools. The recurring problem was that a single model produced an answer, but users couldn't easily determine what was missing or compare another reasoning approach.

I then translated that qualitative information into technical requirements: multiple independent responses, side-by-side comparison, structured evaluation and graceful handling of provider failures.

I didn't have perfect quantitative evidence at that stage, so I deliberately used a prototype as the next information-gathering mechanism.”

---

22. “What did you decide?”

> “I decided not to build the LLM playground as another single-model chat interface.
>
> Instead, I proposed a multi-model comparison architecture where one prompt goes to several providers, their responses are normalized and displayed together, and an evaluation layer helps compare correctness, reasoning, completeness and relevance.
>
> I chose that despite the additional latency, cost and implementation complexity because it solved the learning problem we had identified rather than merely reproducing an existing chatbot experience.”

---

23. “What was the result?”

> “The feature evolved from a basic playground concept into a multi-model learning experience. It is now used by roughly **350 students daily**, and the broader orchestration infrastructure handles thousands of model jobs.
>
> It also became one of the stronger features of the platform and received positive external validation.
>
> More importantly, the outcome validated the original decision: students were using the comparison and evaluation experience rather than us building complexity that nobody wanted.”

---

24. “What did you learn?”

> “I learned that ambiguity doesn't always require waiting for more information.
>
> I first gather enough evidence to understand the customer problem, distinguish reversible decisions from irreversible ones, make a reasoned choice and then use the implementation itself to gather better data.
>
> I also learned that technical sophistication isn't valuable by itself. The multi-model system only made sense because it directly addressed the student problem we had identified.”

High-frequency Behavioral Variants

25. “Tell me about a failure.”

Use **Aasma Timestamp Failure**.

> “I made a design mistake in a Jira/GitHub synchronization system. I assumed timestamps were enough to identify new events. That worked in normal testing but failed when multiple changes occurred in the same second.
>
> I owned the issue, investigated the API contract, discovered unique event identifiers, redesigned the synchronization logic around them and added targeted concurrency/edge-case tests.
>
> The immediate reliability problem was fixed.

What mattered more was changing my process: I now verify the guarantees behind fields I depend on instead of assuming they mean what their names suggest.”

---

26. “Tell me about a disagreement.”

Use **Aasma Microservices**.

“I disagreed with my manager about whether we should change the architecture of a system suffering from recurring debugging problems. I favored microservices; he was worried about migration and deadline risk.

I used evidence and a POC rather than arguing from preference. His concerns also changed my own proposal, so instead of a full migration I recommended a phased approach.

We improved modularity, caught up over two sprints and still met the deadline.

The lesson was to defend the problem strongly but stay flexible on the implementation.”

---

27. “Tell me about receiving difficult feedback.”

Story: Aasma — Timestamp Failure

“The difficult feedback came when the operations team showed that synchronization logic I had designed could miss rapid updates.

It was uncomfortable because my implementation had passed normal testing, but the feedback was valid: I hadn't tested the concurrency edge case deeply enough.

I didn't defend the implementation. I reproduced the problem, revisited the API documentation, replaced timestamp-only identification with unique event tracking and added targeted tests.

The issue was resolved, and I changed how I test integrations afterward.

I try to treat difficult feedback as information about the work rather than as a judgment about me personally."

---

28. "Tell me about a time you had insufficient information."

Use **USC Dornsife**.

"When we were defining the LLM playground, we didn't have large-scale data telling us what students wanted.

I gathered the best available qualitative evidence, identified that comparison and trust were stronger problems than simple access, and treated the decision as reversible.

I built the multi-model comparison system rather than waiting indefinitely for perfect information.

The feature later reached roughly 350 daily students.

My trade-off was speed of learning versus certainty: waiting provided more theoretical safety, but prototyping gave us real evidence much faster."

---

29. “Tell me about a time you had to learn something extremely quickly.”

Use **USC Dornsife provider integrations**.

> “I had to quickly understand several different LLM provider APIs so we could create one consistent experience.
>
> I focused my learning around what the system needed: authentication, request format, response structure, timeout behavior and failure modes.
>
> I then abstracted those differences behind provider adapters and normalized the output.
>
> That let the rest of our platform treat providers consistently and supported the multi-model orchestration used by hundreds of students.
>
> My learning strategy is to go deep first on the interfaces and failure modes that matter to the product, then broaden from there.”

---

30. “Tell me about a time you improved the performance/reliability of a system.”

Story: USC Dornsife — LLM Orchestration

> “When I introduced multiple LLM providers, a naive implementation would call them sequentially. That would make total response time roughly accumulate across providers, and one timeout could delay the entire experience.

I changed the architecture so independent provider calls ran concurrently. I also added provider-specific timeout and failure handling so one unavailable model would not break the entire comparison.

The trade-off was additional concurrency and error-handling complexity, but it substantially improved the responsiveness and resilience of the user experience.

That infrastructure now supports thousands of model jobs as part of a feature used by roughly 350 students daily."

Technical / Scenario Questions

31. "How would you improve a service whose latency has suddenly increased?"

This is **not STAR**. Think aloud:

"First, I would establish whether the regression is real and define its scope. I'd compare p50, p95 and p99 latency with the previous baseline and determine whether it's affecting all endpoints, one region, one customer type or a specific deployment.

Next I'd correlate the start of the regression with recent code, configuration or infrastructure changes. Then I'd use distributed traces and service metrics to break latency into components—application processing, database queries, downstream services, queues and network time.

> I'd check CPU, memory, thread or connection-pool saturation, database query duration and indexes, cache hit rates, queue depth and downstream error/timeout rates.
>
> If customers are actively impacted, I'd prioritize a **reversible mitigation** such as rolling back a deployment, scaling a bottleneck or temporarily disabling a non-critical expensive feature before completing the root-cause analysis.
>
> Once isolated, I'd fix the underlying issue and add an alert or regression test so we detect it earlier next time.
>
> The trade-off is restoration versus diagnosis: during an incident I restore customer experience first using a safe reversible action, then complete the deeper investigation."

That answer should satisfy an SDE1 interviewer.

---

32. "Design a component/service and explain your reasoning."

If they leave it open, use something you genuinely know:

Example: Multi-Model LLM Orchestration Service

> "I'd first clarify that the service receives one prompt, calls multiple model providers and returns normalized responses while tolerating partial failures.
>
> I'd expose something like `POST /comparisons`. Behind the API gateway, a stateless orchestration service determines which providers to call. Each model is behind a provider adapter so GPT, Claude, Gemini or another provider implements a common interface.

The orchestrator fans the requests out concurrently with per-provider deadlines. I'd use limited retries for transient failures, but not unlimited retries because that increases latency and cost. A circuit breaker can temporarily stop calls to a repeatedly failing provider.

Responses are normalized into a common schema and passed to the evaluation layer. If one provider fails, I'd normally return the successful responses rather than failing the entire request.

For interactive requests, I'd prefer synchronous orchestration with a strict overall deadline. For longer evaluation jobs, I'd move execution to a queue and return a job ID.

Workers remain stateless so they can horizontally autoscale. I'd track p95 latency, provider success rate, token cost and partial-failure rate.

The main trade-off is **quality versus latency/cost**: more models improve comparison breadth but increase expense and system complexity."

That also gives you a design you can defend technically because it comes directly from your experience.

Motivation / Fit

33. "Why this role?"

"I'm looking for an SDE role where I can work on systems whose scale and reliability genuinely matter.

My experience has moved increasingly toward backend engineering, APIs, cloud systems and AI-backed services, and the parts I've enjoyed most are debugging difficult systems, making architectural trade-offs and taking a feature from an ambiguous requirement to something people actually use.

This role appeals to me because it would let me deepen those fundamentals on much larger systems while taking meaningful ownership early in my career.

I'm not looking only for a particular framework or technology. I'm looking for technically difficult problems, real customer impact and an environment where strong engineering judgment matters."

For Amazon, the themes are **scale + ownership + customer impact + growth**.

---

34. "What type of engineering work do you want to do?"

"I'm most interested in backend and distributed-systems work—services, APIs, data flows, reliability and performance.

I also enjoy AI infrastructure, but what interests me there is the engineering around the models: orchestration, evaluation, failure handling, observability and scalable serving rather than simply calling an LLM API.

Long term, I want to become the kind of engineer who can take an ambiguous system problem, understand it end to end and make strong architectural decisions."

---

35. “What are you looking for in your next team?”

> “Three things matter most to me.
>
> First, meaningful ownership—I want to be responsible for real components rather than only isolated tasks.
>
> Second, strong engineers I can learn from through design reviews, code reviews and operational experience.
>
> Third, a team whose work has measurable customer or system impact.
>
> I also value a culture where people can disagree technically, use evidence, make a decision and then commit to executing it together.”

---

36. “What do you want to learn?”

> “The biggest area I want to deepen is distributed systems at production scale.
>
> I've already worked with backend services, cloud infrastructure, concurrent processing and multi-provider orchestration, but I want more exposure to what happens when systems operate at much larger scale—capacity planning, fault tolerance, observability, consistency trade-offs and operational ownership.
>
> I also want to become better at making design decisions where there isn't one obviously correct answer and understanding the long-term operational cost of those choices.”

INTRO:

Hi, I'm Sahaj. I recently completed my Master's in Computer Science at USC, and my experience has mainly been across full-stack, backend, and distributed systems engineering.

Currently, I'm working as a Software Engineer at USC Dornsife, where I build backend services and AI orchestration workflows.

Before that, at Grad8, I worked on an AI-powered grading platform, where I developed Flask APIs and helped translate AI and vision-model outputs into a usable product experience.

I've also worked at Aasma Technology Solutions, first as a Full Stack Developer Intern and later as an SDE, where I worked on cross-platform data integrations and backend systems for an industrial application.

Going forward, I'm especially excited to work on large-scale systems where I can own meaningful components end-to-end and solve reliability, scalability, and performance problems that have real customer impact.